stdcbench

A Benchmark for Small Systems

Philipp Klaus Krause
Albert-Ludwigs-Universität
Freiburg, Germany

ABSTRACT

Benchmark programs are useful for measuring performance. Benchmarks written in C effectively benchmark the performance of a C implementation consisting of hardware, compiler and standard library. For small systems (i.e. systems with just a few KB of memory) three well-known and widely used benchmarks are Whetstone, Dhrystone and Coremark. However, all three have their shortcomings. Whetstone scores depend heavily on the performance of floating-point functions from the standard library. Dhrystone scores depend heavily on the performance of just a few string processing functions from the standard library. Coremark intentionally avoids using the standard library and the scores heavily depend on the performance of matrix multiplications. All three thus highly depend on a single aspect of the C implementation each, so that optimizations targeting that aspect have a huge effect on scores.

stdcbench is a benchmark for small systems that tries to give a more balanced reflection of performance. It is intended to be usable for a wide range of C implementations for small systems. We present the design of stdcbench, and discuss a few benchmark results also in comparison to Dhrystone and Coremark.

CCS CONCEPTS

• **Computer systems organization → Embedded systems**; • **Software and its engineering → Software performance**; Compilers;

KEYWORDS

benchmark, stdcbench, Dhrystone, Coremark, embedded systems, microcontrollers, C

ACM Reference Format:
Philipp Klaus Krause. 2018. stdcbench: A Benchmark for Small Systems. In *SCOPES '18: SCOPES '18: 20th International Workshop on Software and Compilers for Embedded Systems, May 28–30, 2018, Sankt Goar, Germany*. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3207719.3207726

1 INTRODUCTION

Benchmark programs are useful for measuring and comparing performance, both wrt. to analyzing existing systems and in making design choices for future systems. Benchmarks written in C effectively benchmark the performance of a C implementation consisting of hardware, compiler and standard library. Few benchmarks in use today are suitable for small systems (systems that have just a few KB of memory). Typically, small systems have different types of memory: Code memory that need not be easily writeable at run-time, and thus is used mostly for storing code and constant data. Typical examples of code memory are EPROM and Flash. Data that needs to be written at run-time is usually stored in data memory. Typical examples include DRAM and SRAM. In most small systems, there is substantially more code memory than data memory. Thus benchmarks for small systems should restrict their use of data memory to a minimum, but can make relatively liberal use of code memory to ensure that a multitude of different code is executed during the benchmark.

Depending on where the code executed, and the input data used during the benchmark originates, a benchmark is categorized as *synthetic* (explicitly written for the benchmark, usually to represent some desired statistical properties of code), *application-based* (taken from real-world applications) or *derived* (from simplified real-world applications).

2 EXISTING BENCHMARKS

While many benchmarks have been developed, only few are relevant for small systems with just a few KB of memory.

Whetstone [1] is one of the earliest benchmarks. It was designed as a synthetic benchmark based on an instruction mix representative for Algol 60 programs [2], which led to a strong emphasis on floating-point performance [3], especially on the quality of the implementation of floating-point mathematical functions in the C standard library. Depending so strongly on floating-point performance would be very unusual for typical programs on small systems today. Due to its low data memory usage of about 1 KB, the benchmark can be executed on small systems.

Dhrystone [4] is another classic synthetic benchmark. It was designed based on a multitude of data on statistical properties from systems programming. However, the interaction of the individual statements and the effects of optimizing compilers had not been sufficiently considered. The resulting issues have been fixed in Version 2 [5], both by modifications to the benchmark and by stating rules for the use of Dhrystone. Dhrystone scores depend heavily on the quality of a few string processing functions from the C standard library. Typically, Dhrystone spends about 30% of the time in the two functions `strcpy()` and `strcmp()` [6]. The data memory usage on 8/16-bit systems is about 5 KB, and about 12 KB for 32-bit systems. This makes it possible to run Dhrystone on many small systems, but excludes some popular microcontrollers that have lower data memory limits, such as the SiLabs EFM8 family (the biggest devices have 4.25 KB of data memory).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SCOPES 2018, May 28–30, 2018, Sankt Goar, Germany
© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.
ACM ISBN 978-1-4503-5780-7/18/05…$15.00
https://doi.org/10.1145/3207719.3207726

Coremark [7] is a newer, derived benchmark. Its main aspects are relevant to embedded systems - lists, state machine, integer matrices. It has been embraced by the Embedded Microprocessor Benchmark Consortium (EEMBC) and been aggressively positioned as a replacement for Dhrystone [8]. Coremark uses mostly 32-bit operations, which is atypical for programs designed to run on small systems including 8/16-bit systems. The integer matrix operations in Coremark have the strongest influence on the score. We found that Coremark scores on small systems mostly just reflect the speed of multiplications - especially on systems that do not have a hardware unit for 32-bit multiplications. Coremark deliberately makes no use of the C standard library, since one of the design goals for Coremark is to keep the influence of the compiler and library on the scores low (and thus make the score more representative of the hardware's capabilities) [9]. However, this is not representative of real-world C programs. The non-free license used for Coremark reduces its suitability - upon release of a new version of Coremark reproducing results from older versions becomes illegal. Coremark uses about 2.5 KB of data memory.

The Standard Performance Evaluation Corporation (SPEC) has released a series of benchmarks since the mid 1990s. While those are considered excellent benchmarks for general-purpose computers, the workloads are not representative for embedded systems [10] and the memory requirements, both in data and code memory are far beyond the capabilities of small systems.

MiBench [10] was designed as an alternative to the non-free EEMBC benchmarks and the SPEC benchmarks, which are not representative of of embedded applications. However, being designed for larger embedded systems, it needs Megabytes of memory and thus is not suitable for small systems.

3 STDCBENCH DESIGN

stdcbench is aimed at small systems programmed in C, should keep the strengths of existing benchmarks for these systems while avoiding their weaknesses, leading to the following design goals:

- Requires no more than 4 KB of data memory
- Requires no more than 32 KB of code memory
- Does not depend on non-standard features in C implementations
- Does not require standard features missing in many C implementations
- Does not require features not required by the standard for freestanding implementations of C90
- Can make use of C99 and C11 features where available
- Can benchmark features of the C library that are useful, but not mandatory for freestanding C implementations
- Gives a balanced reflection of performance (no single functionality should dominate the results)
- Gives a single number as final score
- Verifies correctness of results
- Uses code from or similar to real-world applications
- Free software

stdcbench has a modular design. Scores are calculated for individual modules, and summed to obtain the final score. Individual modules can be disabled for implementations that can not support them (e.g. due to lack of memory or support of certain standard features). In current stdcbench 0.4, two modules exist: c90base and c90lib. The modules have been balanced to contribute about evenly to the final score on many systems. For the future, the addition of a module using floating-point computations is planned, though compared to c90base and c90lib, it will be given lower weight for the final score (reflecting that floating-point computations are not a common task for small systems).

Both c90base and c90lib consists of submodules. Computations from all submodules are done in a loop, then the number of loop iterations per time unit gives the score for the module. Data used in the submodules is passed via `volatile` variables to ensure that computations are not done at compile-time. Each submodule verifies the results of the computation and emits an error message when this fails.

c90base benchmarks a commonly-implemented subset of what the standard requires for freestanding implementations of C90. It intentionally makes no use of floating-point types, the C standard library and does not use much stack space (so it can be used even on architectures with an 8-bit stack pointer, such as the MCS-51). It consists of three submodules. The first submodule executes a decompression algorithm based on Huffman and RLE coding. Two different implementations of the Huffman decoding (iterative and recursive) are used and the results compared for the verification. The decompression algorithms have been taken from real-world code written for small systems and at a lower level exercise control-flow constructs (including function pointers) and bitwise operations. The second submodule is synthetic and does operations on integer matrices. At a lower level, it mostly benchmarks loops and integer multiplications. The third submodule executes an insertion sort algorithm taken from real-world code written for small systems. At a lower level, it mostly benchmarks control-flow constructs, indirection (i.e. data accesses via pointers) and comparisons.

c90lib benchmarks the standard library. It intentionally does not use floating-point types. It consist of two submodules which together exercise a large part of the C standard library. The first submodule computes the linear lnlc-width of a graph. The algorithm has been taken from a real-world library (which was written to be useable on small systems, but is mostly used on bigger systems). At a lower level it mostly exercises memory-handling standard library functions. The second module is a simple peephole optimizer obtained by simplifying the peephole optimizer from the Small Device C Compiler [11]. At a lower level it mostly exercises the string handling and memory management standard library functions.

4 EXPERIMENTS

We have compiled and executed stdcbench 0.4 for various C implementations. The small systems used were based on the 8-bit MCS-51 and STM8 architectures and the 32-bit ARM architecture. For easy reproduction of our results, the ports to various boards are included as examples distributed with the stdcbench source code.

The EZ-USB FX2LP USB2.0 board has Cypress CY7C68013A processor with a core based on the MCS-51 architecture, which we ran at the maximum speed of 48 Mhz. The fastest instructions take 4 clock cycles to execute, but many instructions take longer, which is typical for older implementaion of the MCS-51 architecture still in common use. The C8051F V2.1 board has a SiLabs C8051F120 processor with a core based on the MCS-51 architecture, which

we ran at a speed of 98 Mhz (the maximum that can be achieved using the internal oscillator; otherwise the maxium is 100 Mhz). Most instructions execute in a single clock cycle, which is typical of current implementations of the MCS-51 architecture.

The STM8AF board from the STM8A Discovery kit has an STMicroelectronics STM8AF5288 processor with a core based on the STM8 architecture, which we ran at 16 Mhz (the maximum that can be achieved using the internal oscillator; otherwise the maximum is 24 Mhz). At this speed the flash memory requires no wait states. The fastest instructions take 1 clock cycle to execute.

The STM32F0-Discovery board has an STMicroelectronics STM32F051R8 processor with an ARM Cortex-M0 core, which we ran at the maximum speed of 48 Mhz. At this speed the Flash memory requires 1 wait state for accesses. The STM32F302 Nucleo-64 NUCLEO-F320R8 board has an STMicroelectronics STM32F302R8 processor with an ARM Cortex-M4 core, which we ran at 64 Mhz (the maximum that can be achieved using the internal oscillator; otherwise the maximum is 72 Mhz). At this speed the Flash memory requires 2 wait states for accesses.

Processor	Architecture	data memory	code memory
C8051F120	MCS-51	8.25 KB	128 KB
CY7C68013A	MCS-51	16.25 KB	
STM8AF5288	STM8	6 KB	64 KB
STM32F051R8	ARM	8 KB	64 KB
STM32F302R8	ARM	16 KB	64 KB

For 8-bit architectures we mostly used the Small Device C Compiler [11], version 3.7.0 with the included libraries. This compiler targets some non-orthogonal architectures, which are hard to target in compilers aimed a bigger systems. It has a few unusual optimizations particularly useful for its targets [12–14]. For the 32-bit architectures we used the GNU Compiler Collection, version 6.3.1 with newlib-nano (from the gcc-arm-none-eabi and newlib-arm-none-eabi Debian packages). To look into the effect of varying compiler / compiler options and libraries for some given hardware, we did additional tests for the STM8AF system: We compiled stdcbench using different compilers (Cosmic 4.4.7, Raisonance 2.62.17.263, SDCC 3.5.0, SDCC 3.6.0, SDCC 3.7.0) and optimization goals (code size, code speed, balanced). For SDCC, we used `-mmcs51 --model-large --stack-auto --max-allocs-per-node 10000` options for MCS-51 and `-mstm8 --max-allocs-per-node 10000` options for STM8. For GCC, we always used `-pedantic -std=c99 -O2 --specs=nano.specs -mlittle-endian -mthumb -ffunction-sections -fdata-sections` options.

For comparison, we also obtained Dhrystone 2.1 and Coremark 1.0 results.

5 BENCHMARK RESULTS

Our first benchmark results can be found in the table on the following page. Missing entries are due to limitations in the C implementation (memory limits for the MCS-51, standard library completeness for Cosmic, SDCC 3.5.0 and 3.6.0, incomplete implementation of the C standard for Raisonance). While the results mostly speak for themselves, we want to point out a few interesting aspects.

The goals for compiler optimizations are not always in line with the results. One can see that the Raisonance compiler achieves a higher score in stdcbench when optimizing for size than when optimizing for speed. Similarly, SDCC 3.5.0 and 3.6.0 achieve higher Dhrystone scores when optimizing for code size than when optimizing for speed.

When SDCC authors noticed in 3.6.0 that their Dhrystone and Coremark scores were substantially below those of other compilers, despite SDCC generating better or equivalent code for large parts of these benchmarks, they analyzed those two benchmarks, and addressed the issue (by providing faster implementations of some string functions in the standard library and introducing a few optimizations aimed at (matrix) multiplications). One can see the huge impact of these few changes in scores more than doubling from SDCC 3.6.0 to 3.7.0, while the stdcbench scores do not show a corresponding change.

One can also see the effect from the excessive use of 32-bit arithmetic in Coremark: When comparing the 8-bit STM8AF5288 to the 32-bit STM32F302R8, the latter achieves nearly 18 times the Coremark score. On the other hand, the Dhrystone and stdcbench scores are only about 12 times the score of the 8-bit system.

6 CONCLUSION

We have presented stdcbench, a benchmark for C implementations that is suitable for small systems. It does not suffer from some of the weaknesses identified in the Whetstone, Dhrystone and Coremark benchmarks. In its current state it is a suitable replacement for Dhrystone and Coremark. After the future addition of a module for floating-point calculations it could also replace Whetstone. stdcbench can be found at www.stdcbench.org.

This research was supported by Deutsche Forschungsgemeinschaft, Projekt Graphstrukturtheorie im Übersetzerbau, KR 4970/1-1.

REFERENCES

[1] H. J. Curnow and Brian A. Wichmann. A Synthetic Benchmark. *Computer Journal*, 19:43–49, 1976.
[2] Brian A. Wichmann. Some statistics from ALGOL programs, August 1970.
[3] Reinhold P. Weicker. An Overview of Common Benchmarks. *Computer*, 23(12):65–75, December 1990.
[4] Reinhold P. Weicker. Dhrystone: a synthetic systems programming benchmark. *Comms. of the ACM*, 27:1013–1030, October 1984.
[5] Reinhold P. Weicker. Dhrystone Benchmark: Rationale for Version 2 and Measurement Rules. *SIGPLAN Notices*, 23:49–62, August 1988.
[6] Reinhold P. Weicker. Understanding Variations in Dhrystone Performance. *Microprocessor Report*, pages 16–17, May 1989.
[7] Shay Gal-On and Markus Levy. Exploring Coremark - A Benchmark Maximizing Simplicity and Efficacy, May 2012.
[8] Alan R. Weiss. Dhrystone Benchmark - History, Analysis, "Scores" and Recommendations. November 2002.
[9] Shay Gal-On and Markus Levy. Creating portable, repeatable, realistic benchmarks for embedded systems and the challenges thereof. In *Proceedings of the 13th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, Tools and Theory for Embedded Systems*, LCTES '12, pages 149–152, New York, NY, USA, 2012. ACM.
[10] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown. Mibench: A free, commercially representative embedded benchmark suite. In *Proceedings of the Workload Characterization, 2001. WWC-4. 2001 IEEE International Workshop*, WWC '01, pages 3–14, Washington, DC, USA, 2001. IEEE Computer Society.
[11] Sandeep Dutta. Anatomy of a Compiler. *Circuit Cellar*, 121:30–35, 2000.
[12] Philipp K. Krause. Optimal Register Allocation in Polynomial Time. In *Compiler Construction - 22nd International Conference, CC 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013. Proceedings*, volume 7791 of *LNCS*, pages 1–20. Springer, 2013.
[13] Philipp K. Krause. Bytewise Register Allocation. In *Proceedings of the 18th International Workshop on Software and Compilers for Embedded Systems*, SCOPES '15, pages 22–27. ACM, 2015.
[14] Philipp K. Krause. Optimal Placement of Bank Selection Instructions in Polynomial Time. In *Proceedings of the 16th International Workshop on Software and Compilers for Embedded Systems*, M-SCOPES '13, pages 23–30. ACM, 2013.

Processor	Clock	Compiler	Extra options	Dhrystone size	Dhrystones per second	Coremark size	Coremark iterations per second	stdcbench size	stdcbench c90base score	stdcbench c90lib score	stdcbench total score
CY7C68013A	48 Mhz	SDCC 3.7.0	–opt-code-speed	10618[1]	798[1]			13379	12		12
C8051F120	98 Mhz	SDCC 3.7.0	–opt-code-speed	10590[1]	6587[1]			13376	95		95
STM8AF5288	16 Mhz	Cosmic 4.4.7	[optimize for speed]	8483	8558	14547	3.814	8610	122		122
STM8AF5288	16 Mhz	Cosmic 4.4.7		8457	8852	14084	3.343	8609	122		122
STM8AF5288	16 Mhz	Cosmic 4.4.7	[optimize for size]	8330	8632	13692	3.331	8576	116		116
STM8AF5288	16 Mhz	R. 2.62.17.263	[optimize for speed]	5661	8090	11226	3.655	5983	74		74
STM8AF5288	16 Mhz	R. 2.62.17.263	[optimize for size]	5659	8061	10210	2.529	5117	109		109
STM8AF5288	16 Mhz	SDCC 3.5.0	–opt-code-speed	8173	4240	15881	2.978	6092	120		120
STM8AF5288	16 Mhz	SDCC 3.5.0		8173	4240	15914	2.900	6094	117		117
STM8AF5288	16 Mhz	SDCC 3.5.0	–opt-code-size	8118	4250	15826	2.889	6073	118		118
STM8AF5288	16 Mhz	SDCC 3.6.0	–opt-code-speed	8191	4574	15967	3.002	6141	122		122
STM8AF5288	16 Mhz	SDCC 3.6.0		8191	4574	16006	2.922	6145	121		121
STM8AF5288	16 Mhz	SDCC 3.6.0	–opt-code-size	8135	4576	15914	2.915	6126	121		121
STM8AF5288	16 Mhz	SDCC 3.7.0	–opt-code-speed	8025	10233	14735	6.073	21219	109	88	197
STM8AF5288	16 Mhz	SDCC 3.7.0		8016	9765	15024	5.653	21261	106	87	193
STM8AF5288	16 Mhz	SDCC 3.7.0	–opt-code-size	7966	9704	14969	5.608	21110	107	87	194
STM32F051R8	48 Mhz	GCC 6.3.1	-mcpu=cortex-m0			14388	86.083	28096	1141	651	1792
STM32F302R8	64 Mhz	GCC 6.3.1	-mcpu=cortex-m4	14752	119674	13164	108.834	27800	1693	864	2557
STM8AF5288	16 Mhz	IAR 3.10.1.201	[optimize for speed]	10440	7144	18468	5.022	27304	189	100	289
STM8AF5288	16 Mhz	IAR 3.10.1.201		10155	6021	14159	3.398	24674	142	73	215
STM8AF5288	16 Mhz	IAR 3.10.1.201	[optimize for size]	10149	5733	14007	2.652	24297	116	71	187
STM8S208MB	24 Mhz	SDCC 3.7.0	–opt-code-speed	8050	13098	14760	8.365	21244	147	119	266
STM32F103RB	36 Mhz	GCC 6.3.1	-mcpu=cortex-m3	14356	71428	13104	74.055	27196	1158	575	1733
STM32L073RZ	32 Mhz	GCC 6.3.1	-mcpu=cortex-m0plus			14404	57.151	28112	717	414	1131

[1]benchmark had to be compiled for MCS-51 without the –stack-auto option, this substantially reduces stack and code memory consumption at the cost of making all functions non-reentrant (SDCC targeting MCS-51 is not a standard-compliant compiler wrt. reentrancy without –stack-auto)